Motor Speed Control with a PI Controller and Anti-Windup

Lab 3 – digital control of the T-Bot motors (and Lab 4 line following)

Embedded Real-Time Control laboratory – M.Sc. Control Systems Engineering, University of Padova
Author: Kiavash Rabbani Soudabi (ID 2141822) – Course held by Prof. Alberto Morato – 2025

Purpose

Implement a PID controller for the T-Bot drive motors: measure the wheel speed from the quadrature encoders, close the loop around the motor voltage, and command a reference speed from the robot keypad, with telemetry for tuning in MATLAB.

What I implemented

- **Speed measurement:** I read the TIM3/TIM4 encoder counters and computed the per-period count difference, explicitly handling counter overflow and underflow so that the direction and the magnitude stay correct; counts are converted to rad/s with the encoder/gear scaling.
- **Digital PI controller** executed in the timer period-elapsed callback at a fixed sampling time of `TS = 10 ms`: the error is evaluated in rad/s, with the proportional and integral terms $P = 0.871$, $I = 7.5563$, and the integral action updated as $I \mathrel{+}= I\,T_s\,e$.
- **Anti-windup:** conditional integration – if the output saturates *and* the error would worsen the saturation, the integral step is removed, which keeps the integrator usable after long saturation episodes.
- **Output conditioning and drive:** the control signal (volts) is limited to $\pm 6\,\mathrm{V}$, scaled to PWM duty against the $12\,\mathrm{V}$ battery and the timer ARR value, and applied with two drive modes – `ForwardCoast` (single-channel PWM) and `ForwardBreak` (complementary PWM for braking).
- **Operator interface:** the reference speed is entered on the keypad (up to three digits followed by `#`, decoded with `atoi`), and the keypad decoding/handling is interrupt- and buffer-based.
- **Telemetry:** the control signal and the measured quantities are streamed through the course datalogger for plotting and tuning in MATLAB.
- **Lab 4 (line following)** reuses this exact motor-control base and adds the line sensor handling – storing the sensor pattern (`StoreBinary`) and computing the lateral error used by the steering law. Lab 4 has no separate report; its implementation lives in the same code base.

Results

The speed loop tracks the commanded reference on both motors, with the measured step response and the control signal recorded by the datalogger and analysed in MATLAB. The report documents the controller structure, the gains, the anti-windup logic and the measured tracking behaviour. The same base was extended for the autonomous line-following task of Lab 4.

Repository contents

`report/Embedded-Report-lab3.pdf` (submitted report) · `src/user_code_main.c` (USER CODE sections of my `main.c` – speed measurement, PI controller with anti-windup, drive functions, keypad command handling; the course framework around them is not redistributed) · `docs/` (this overview and report pages).

Skills demonstrated

Digital control implementation (PI, fixed sampling time, anti-windup), quadrature encoder interfacing and overflow-safe speed estimation, PWM motor drives including braking mode, embedded real-time scheduling (timer interrupt control loop), measurement, logging and tuning with MATLAB, bare-metal C on ARM Cortex-M.